



Energy-Aware Neural Architecture Search: Leveraging Genetic Algorithms for Balancing Performance and Consumption

Antoine Gratia, Paul Temple, Gilles Perrouin, Pierre-Yves Schobbens

► To cite this version:

Antoine Gratia, Paul Temple, Gilles Perrouin, Pierre-Yves Schobbens. Energy-Aware Neural Architecture Search: Leveraging Genetic Algorithms for Balancing Performance and Consumption. WIVACE 2024 - XVIII International Workshop on Artificial Life and Evolutionary Computation, Sep 2024, Namur, Belgium. pp.1-13. <hal-04957679>

HAL Id: hal-04957679

<https://inria.hal.science/hal-04957679v1>

Submitted on 19 Feb 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Energy-Aware Neural Architecture Search: Leveraging Genetic Algorithms for Balancing Performance and Consumption

Antoine Gratia¹[0009-0009-7234-3110], Paul Temple²[0000-0002-8276-0593],
Pierre-Yves Schobbens¹[0000-0001-8677-4485] and Gilles
Perrouin¹[0000-0002-8431-0377]

¹ PRECISE/NaDI, Computer Science Faculty, University of Namur, Belgium

² University of Rennes, CNRS, Inria, IRISA, Rennes, France

Abstract. In recent years, the field of deep learning has been propelled forward by increasingly complex and resource-intensive neural network models. Despite their impressive performance, their training consumes much energy, incurring significant environmental impacts. Even more so, when neural architecture search (NAS) methods train hundreds of thousands of models to benchmark optimization. We offer to create greener benchmarks using genetic algorithms. We leverage our previous CNNGen approach producing random CNN topologies based on our context-free grammar. We then use the NSGA-II genetic algorithm to create topologies balancing performance and energy consumption. We rely on machine-learning-based predictors that estimate candidates' performance and energy consumption to avoid training during generations, saving significant computational costs. This paper reports on our experiments and discusses further developments.

Keywords: Genetic Algorithms · Neural Architecture Search · Green AI.

1 Introduction

Deep learning has made remarkable strides, driving diverse applications such as image and speech recognition [6, 9], natural language processing [16], and autonomous vehicles. These advancements are largely attributed to the increasing size and complexity of models [2, 17]. Yet, we can wonder to what extent we need to make the models more complex if the gain in performance is marginal. Conversely, can we shrink the deep learning models down while maintaining performance? Identifying the optimal architecture for a given task is a challenging and time-consuming endeavor. To address this, Neural Architecture Search (NAS) [8] formulates an iterative optimization problem to design neural network models that achieve the best performance. Datasets have been developed to benchmark generated models and validate NAS optimization approaches. NASBench-101 [19] is probably the most popular and can generate thousands

of unique architectures by systematically combining connections and network functions within predefined cells. However, exploring the space of neural architectures is resource-intensive and has a significant carbon footprint. For instance, training LLaMA [16] involves thousands of high-end GPUs, resulting in an estimated carbon footprint of 173 tCO₂eq (*i.e.*, tons of CO₂ equivalent). To put this into perspective, training a single Transformer model for NLP tasks can emit as much carbon as the lifetime emissions of five cars [15]. Despite this, current NAS techniques do not account for resource consumption and carbon footprint in their algorithms and datasets, focusing solely on performance (accuracy). Moreover, we argue that NAS benchmarks, such as NASBench-101, lack architectural diversity. These benchmarks are based on an initial architecture that already predicts well (above 80% accuracy [19]), allowing only limited variations within cells. Consequently, most generated architectures exhibit training accuracies that exceed 99% on CIFAR-10 when trained for 108 epochs. Additionally, reproducing results is challenging due to the unavailability of essential artifacts, such as fully trainable model code. Our previous work, CNNGen [5], a convolutional neural network (CNN) generator based on a domain-specific language (DSL) and a context-free grammar, was designed to construct CNN models from scratch and provide the expected artifacts (*e.g.*, models, parameters, performances and energy consumption, and code). In addition, we have proposed several predictors for energy consumption and performance. They rely on the description of the models to predict the expected accuracy and/or energy consumption without training the models. Thus, we can explore many models for a limited cost. In this work, we propose using genetic algorithms (GA) to generate new CNN models that balance energy consumption and performance. The GA iteratively evolves a population of CNN models, selecting and recombining them based on a fitness function that considers both energy efficiency and accuracy. Note that we rely again on the estimators to evaluate the performance and energy consumption of new population members. This approach not only fosters greater architectural diversity, but also promotes the development of more sustainable CNN exploration and optimization methods. Concretely, we experimented with the off-the-shelf NSGA-II algorithm. Our results show that our workflow can optimize a topology after five hundred generations, with exhibiting certain CNNs with a slightly better performance and significant decrease in energy consumption. We also discuss the role of predictors and the complexity of predicting performance and accuracy in such a complex CNN topological space.

The rest of the paper is as follows. Section 2 presents the main concepts of neural architecture search and our previous work, which allows us to produce diversified topologies. Section 3 describes the genetic algorithm we experimented with and how it operates with our CNN models as well as our accuracy and energy consumption predictors, while Section 4 reports on experiments. Section 5 offers a critical reflection on predictor performance, while Section 6 wraps up the paper.

2 Background

Neural Architecture Search (NAS) focuses on automating the design of neural network models, focusing on search space, search strategy, and performance estimation. Search strategies include random search, Bayesian optimization, evolutionary methods, and reinforcement learning [11, 14, 20]. Performance estimation techniques, such as learning curve extrapolation, surrogate models or network morphisms, aim to reduce computational costs [1, 4, 8, 12, 13]. AutoML, exemplified by Auto-Keras [7, 8], simplifies the usage of machine learning models for individuals with limited expertise. While Auto-Keras starts with an existing architecture and modifies it through morphing, it requires a good starting point and explores a small neighborhood around the current architecture. State-of-the-art NAS techniques [10] also require a reference architecture to start the search. This architectural choice is left to the experts.

Multi-Objective Genetic Algorithms (MOGAs) are optimization techniques that evolve a population of candidate solutions towards optimization strategies such as Pareto-optimal fronts, addressing problems with multiple conflicting objectives. Inspired by natural selection, MOGAs use crossover, mutation, and selection to explore the solution space. They aim to find a diverse set of trade-off solutions rather than a single optimal solution, making them particularly useful in complex scenarios in which objectives cannot be easily aggregated. Techniques like non-dominated sorting and crowding distance ensure convergence towards and maintenance of diverse solutions across optimization strategies. MOGAs have been applied effectively in various fields such as engineering design, economics, and machine learning. As argued previously, we want to combine MOGAs with NAS to generate new predicting models that would both perform with high accuracy and with a reduced energy consumption compared to current popular models.

Domain-Specific Languages (DSLs) are high-level software languages capturing the essence of a particular domain via specialized abstractions [18]. A DSL typically provides a concise, expressive syntax that enables developers to easily and accurately model complex systems and capture domain knowledge with constraints, reducing the risk of errors or inconsistencies. Then, dedicated generators derive artifacts (code, documentation) automatically from these models. DSLs exist for numerous domains (*e.g.*, software engineering, biology, or robotics) and at various abstraction levels (from programming GPUs to language modeling).

CNNGen [5] is a novel dataset-agnostic approach to generate energy-aware benchmarks for CNNs. CNNGen captures the CNN space using context-free grammar and specifies them with a DSL. CNNGen starts with a description of the topology of the CNN model to be created, which is called a description sentence. The topology refers to the succession of layers that will be stacked (but without any specification of the parameters). As we said before, CNNGen comes with domain-specific rules regarding the succession of layers preventing the generation of unrealistic models. Once the topology is generated, a second phase assigns coherent parameters to every single layer. Along with the description sentence, CNNGen generates various artifacts: an image representation (in

PNG format) of the model associated with the sentence, a TensorFlow runnable code, and a report containing the performance and energy consumption of the trained model associated with the description sentence. By generating random description sentences that meet the generation rules, we can create our dataset of CNN models similar to NAS benchmarks but, unlike NAS approaches, it does not necessitate finding a well-performing CNN model first. In the end, if we want to search for a well-performing CNN model, we simply need to guide the exploration of the gigantic CNN topology with a tool implementing an optimization strategy. In this work, we used MOGAs to maximize the model’s accuracy and minimize its energy consumption. The problem is that MOGAs need to know the accuracy and energy consumption associated with each model to decide whether they should be kept for future generation. Instead of training each model, we rely on predictors.

Predictors [5] have also been proposed along CNNGen to avoid fully training every single generated CNN model. As stated before, CNNGen reports both the accuracy and the energy consumption of generated models, thus, we proposed two distinct kinds of predictors. The first predictor assesses the accuracy by analyzing the Python code generated by CNNGen, which was designed to run specific CNN models based on their description sentences. This predictor concentrated on the various architectural layers of the CNN and extracted their respective hyperparameters. The primary objective was to identify and learn correlations between the CNN layers and their hyperparameters, as well as the overall performance of the architecture.

The second predictor is a Regressor Decision Tree which analyzes four manually defined features to predict the energy consumption of CNN models. These features are the number of layers, the number of epochs used for training, the floating-point operations per second (FLOPs), and the total number of parameters in the model. In this paper, we considered a random forest for better energy prediction.

Similarly to ours, Mo and Iacca use a surrogate machine learning model to estimate the performance of models to avoid training and accelerate the evolution of the genetic algorithm [13]. Our approach extends this idea to a multi-objective setting. The same authors exploited learning curve estimation still requires (minimal) training of the models to estimate their performance [12], while our work adopts a radical no-training approach during search.

3 Sustainable NAS Framework

Our approach relies on CNNGen to create an initial population of neural networks and uses NSGA-II and our improved predictors to evolve them.

3.1 NSGA-II

NSGA-II (Non-dominated Sorting Genetic Algorithm II) is a popular multi-objective optimization genetic algorithm designed to solve complex problems

with multiple conflicting objectives [3]. The algorithm finds a set of optimal solutions, known as the Pareto front, where no solution can be improved regarding one of the objectives without degrading another. NSGA-II starts by generating an initial population of potential solutions and evaluates their performance on each objective. These solutions are sorted based on dominance: a solution is said to dominate another if it is better in at least one objective and is not worse in any others. The algorithm then uses a ranking system to classify solutions into different non-dominated fronts, with the first front being the best, etc. NSGA-II employs a crowded comparison operator to ensure diversity amongst the solutions. This operator ranks solutions within the same front based on their proximity to other solutions, preferring diversity instead of peaking all the solutions in the same neighborhood. The process involves selection, crossover, and mutation operations to create a new generation of solutions. With each generation, NSGA-II explores the solution space, aiming to find a well-distributed set of high-quality solutions along the Pareto front. The algorithm continues iterating through generations until a stopping criterion is met, such as a maximum number of generations. Currently, our CNN generator approximatively generates 1% of architectures that do not compile. Yet, because of the mutations and crossover, and despite specific precautions, the number of errors, and therefore the number of architectures that do not compile, increases up to 10%. To avoid keeping these architectures in the pool of solutions and further help NSGA-II, we give these architectures the highest possible float value. The final output is a diverse set of solutions representing the trade-offs between the different objectives. Decision-makers simply have to choose any solution among these.

3.2 Improved Predictors

We utilized the predictors proposed by CNNGen [5] but observed that their predictions were not sufficiently accurate. To improve reliability, we modified these predictors by creating a new dataset consisting of 40,000 architectures generated with CNNGen. We also enhanced the preprocessing of input data, optimising transformations and performing a comprehensive grid search to identify better hyperparameters for more accurate performance predictions (i.e. transformer model). For the energy predictor, we applied feature engineering techniques to extract new features that could enhance prediction accuracy. Our analysis identified several key features as particularly relevant, including the number of layers, FLOPs, parameters, convolutional layers, and fully connected layers, as well as the presence or absence of global average pooling, max pooling, and flattening operations. Additionally, we experimented with various machine learning techniques, including decision tree-based methods, support vector regression, and random forests. After evaluating their performance, we concluded that random forest yielded the most accurate predictions, making it the most suitable approach for improving energy estimation in our framework. In the end, regarding accuracy prediction, we have used a transformer model on the Python code associated with a CNN architecture (as done previously in CNNGen) while for

the prediction of the energy consumption, we have used a random forests composed of 50 trees that use the aforementioned retrieved features from the feature engineering study.

4 Experimentations

4.1 Experimental Protocol

Whenever a new CNN model is used, we rely on CNNGen to produce a random description sentence used to generate the associated model. For instance, CNNGen is called to build our initial population that will evolve through genetic algorithms. To assess fitness, we employ performance and energy consumption predictors to avoid the need for training architectures. These predictors evaluate both the performance (accuracy) of architectures and energy consumption. Both elements are combined in the fitness function to form a multi-objective optimization problem to guide the search process. But, these predictors need to be trained first. Again, we randomly generated 40,000 models from CNNGen. We trained them on CIFAR-10 datasets for 20 epochs. Then, our 40,000 models are divided into 80% for training the predictors, and the remaining 20% are used for evaluating the predictors. Among the 80% dedicated to training, 20% of them are used for validation. As stated before, we only consider NSGA-II. We describe their parameterization and present the results hereafter.

4.2 NSGA-II

NSGA-II generates a Pareto front of non-dominated solutions regarding the different objectives. Thus, in our case, the resulting set of CNN models presents different trade-offs between accuracy and energy consumption, leaving the final choice to the end-user. To conduct this experiment, we used the version of NSGA-II provided with JMetal, a Java library, in its version 6.2.2³. We set the size of the initial population and the offspring sizes to 10. We ran the genetic optimization for 500 generations. We have also set the cross-over and mutation rates at 60% and 10% respectively.

4.3 Predictors parameterisation

As said previously, we rely on a Transformer model based on Python code associated with CNN models to predict the performance (*i.e.*, the accuracy) of models generated via CNNGen. This predictor is the same as the one used in our previous work [5]. Regarding the predictor for energy consumption, we have used a random forest composed of 50 decision trees. These trees are limited to a maximum depth of 10 and use a mean square error criterion for output. The possible splits use the features listed in Section 3.2, defined after using a feature engineering analysis.

³ <https://github.com/MOEAFramework/JMetal-Plugin/releases/tag/v6.2.2>

4.4 Results

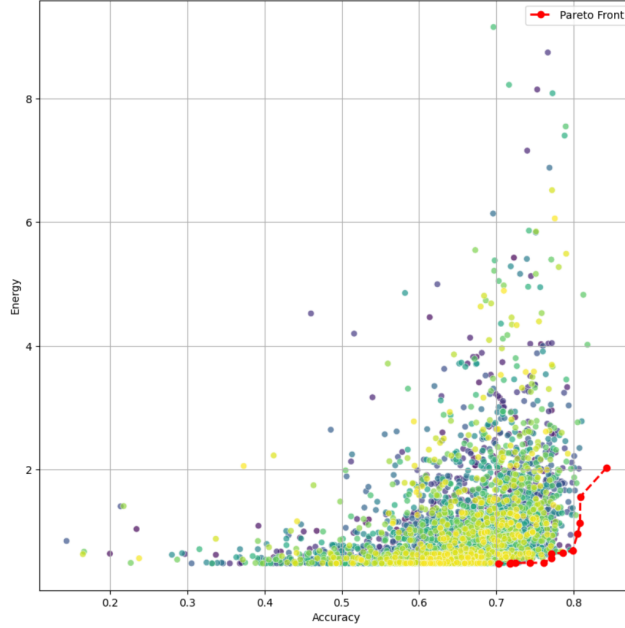


Fig. 1: Results of the NSGA-II optimization process for 500 generations. The X-axis represents the predicted accuracy for all the architectures. The Y-axis represents the predicted value of energy (CO_2 equivalent in grams). The red curve (at the bottom-right) reports the Pareto front.

Figure 1 presents the results of one NSGA-II optimization run. This scatter plot presents the trade-off between accuracy and energy consumption for various CNN architectures generated using CNNGen. Each point represents an individual architecture, with accuracy plotted on the x-axis and energy consumption on the y-axis. The red curve highlights the Pareto front, which consists of architectures that achieve the best possible trade-off between these two competing objectives. From the distribution of points, we observe that architectures with lower accuracy tend to have lower energy consumption, forming a dense cluster at the bottom of the plot. As accuracy increases, energy consumption generally increases, suggesting that more accurate models require higher computational resources. However, the spread of points indicates significant variation in energy consumption for architectures with similar accuracy, implying that some models are more energy-efficient than others at the same performance level. The Pareto front follows a characteristic upward curve, where architectures along this frontier represent the optimal balance between accuracy and energy. Any point not on the Pareto front is a suboptimal solution, meaning that another architecture

exists with either a higher accuracy and the same energy consumption, or a lower energy consumption and the same accuracy, or being better at both objectives at the same time. The shape of the front suggests that beyond a certain accuracy threshold, increasing performance comes with a disproportionately higher energy cost. This observation highlights the challenge of designing CNN architectures that maximize accuracy while remaining energy-efficient. Overall, this analysis emphasizes the importance of multi-objective optimization in neural architecture search. While higher accuracy is desirable, the energy cost must be carefully considered, especially for applications where computational efficiency is crucial.

To complete this general analysis, Tables 1-3 present individual performance and consumption of our architectures at generation 0, 250, and 500.

Name	Pred. Accuracy	True Accuracy	Pred. Energy	True Energy
Architecture 0	0.74	0.69	1.1	1.06
Architecture 1	0.65	0.63	0.63	0.73
Architecture 2	NaN	NaN	NaN	NaN
Architecture 3	0.57	0.56	1.03	0.96
Architecture 4	0.44	0.59	0.54	0.58
Architecture 5	0.69	0.48	2.69	3.03
Architecture 6	0.58	0.66	1.19	1.33
Architecture 7	0.73	0.63	1.10	1.16
Architecture 8	0.71	0.60	3.07	2.95
Architecture 9	0.72	0.67	0.64	0.62

Table 1: Generation 0 - CO² equivalent in grams. NaN: The architecture could not compile.

Name	Pred. Accuracy	True Accuracy	Pred. Energy	True Energy
Architecture 0	0.70	0.66	0.56	0.57
Architecture 1	0.49	0.41	0.48	0.39
Architecture 2	0.67	0.56	0.86	1.12
Architecture 3	0.77	0.71	0.64	0.71
Architecture 4	0.61	0.63	0.96	0.98
Architecture 5	0.46	0.53	0.66	0.82
Architecture 6	0.66	0.55	0.56	0.57
Architecture 7	0.64	0.52	0.72	0.76
Architecture 8	0.75	0.73	1.69	1.89
Architecture 9	0.68	0.67	1.44	1.51

Table 2: Generation 250 - CO² equivalent in grams. NaN: The architecture could not compile.

We can observe that, on average, the true accuracy of generated models slightly decreases (0.61 for generation 0 vs 0.58 for generation 500), representing a 5% decrease. However, energy consumption decreases more significantly (1.38 for generation 0 vs 0.84 for generation 500), representing a decrease of 39%. If we look at individuals, we find interesting compromises. For example, architecture 7 in generation 500 achieves almost the same performance as architecture 8 in generation 250 for half of the energy consumed. This demonstrates the capability of our approach to balance accuracy and energy consumption.

Name	Pred. Accuracy	True Accuracy	Pred. Energy	True Energy
Architecture 0	NaN	NaN	NaN	NaN
Architecture 1	0.52	0.50	0.53	0.50
Architecture 2	0.61	0.59	0.48	0.39
Architecture 3	0.72	0.49	1.08	1.09
Architecture 4	0.74	0.49	0.98	0.99
Architecture 5	0.62	0.65	1.05	1.09
Architecture 6	NaN	NaN	NaN	NaN
Architecture 7	0.73	0.72	0.91	0.91
Architecture 8	0.68	0.62	0.82	0.95
Architecture 9	NaN	NaN	NaN	NaN

Table 3: Generation 500 - CO² equivalent in grams. NaN: The architecture could not compile.

5 Discussion & threats to validity

5.1 Discussion

In the previous section, we presented our results for our sustainable NAS workflow. In particular, we integrated performance and energy predictors into the workflow to reduce computational costs. While we conducted standard procedures to train and evaluate these predictors, the genetic algorithm may focus on generating more and more architectures that were outside the data distribution of our initial training set. Since the overall optimization goal is to focus on only a tiny subset of the possible CNN architectures that CNNGen can produce, it is not surprising that, over time, the data distribution drifts toward new architectures that were difficult to grasp with random generation. Thus, we assume that the more generations NSGA-II produces, the less accurate our predictors will be if they remain as-is. Yet, this can be improved in different ways. For instance, using more training data to get better predictors. This may require tuning the CNNGen generation of description sentences to look for architectures with higher accuracies or lower energy consumption. But, training more CNN models to train our predictors is not a panacea: *i*) it requires more resources

which negatively impacts sustainability, *ii*) cross-over and mutations will still contribute to out-of-distribution topologies. A more promising idea would be to retrain our estimators regularly after a specific number of generations. For instance, for every 200 NSGA-II generations, we launch a retraining procedure on our predictors to follow the evolution of the generated CNN architectures. This way, the predictors will follow the drift in the data distribution which may control the difference between the predicted and actual values of both accuracy and energy consumption. Of course, other strategies could be used, such as active learning or continuous retraining.

The analysis of the tables across different generations reveals that the predictors used for estimating both accuracy and energy consumption are not entirely reliable. In Generation 0 1, while some predictions are relatively close to the true values, discrepancies are noticeable, such as in Architecture 5, where the predicted precision (0.69) is significantly higher than the true accuracy (0.48). Additionally, some architectures, like Architecture 4, exhibit an inversion where the predicted accuracy is lower than the actual value, suggesting inconsistencies in the predictor’s estimations. This may lower the confidence of using the proposed architectures by final users. However, we can observe that the predictions remain, mainly, in a 5% error envelope, the estimations on the energy consumption seem to be less stable (ranging from 2% to 15%), showing once again that further effort should be made in considering energy consumption as one of the goal of optimizing CNN (and more generally deep learning) architectures. As the generations progress, the presence of NaN values in Generation 500 (see Table 3) highlights another critical issue: the increasing occurrence of architectures that cannot be compiled. This could be a consequence of the mutation and crossover mechanisms of NSGA-II.

While Tables 1 and 2 show only 1 architecture that could not be compiled (over 20 architectures), Table 3 reports 4 of such architectures. Our cross-over and mutation operators strive to limit topology-breaking changes. Yet the effect of randomness cannot be completely avoided, notably regarding the determination of some hyperparameters.

Besides, the tables suggest that the energy estimations from the predictors also suffer from inaccuracies. In several cases, the predicted energy consumption does not align well with the true values, such as in Generation 250 (see Table 2), where Architecture 2 was predicted to have an energy consumption of 0.86 but in reality consumed 1.12. This discrepancy indicates that the energy predictor, but it also applies to the accuracy predictor, requires further refinement to ensure reliability in guiding the search process. Another concerning trend is that as the search progresses, the performance of certain architectures deteriorates instead of improving. In Table 3, some architectures (*e.g.*, Architecture 3 and Architecture 4) exhibit a drop in true accuracy despite relatively high predictions. This further reinforces the risk that mutations and crossovers are creating architectures that lie outside the training distribution of the predictor, leading to increasingly unreliable predictions.

5.2 Threats to Validity

5.3 Internal validity

Threats to internal validity concern the implementation and experimental protocol. Regarding NSGA-II we used JMetal, a popular GA framework. While our optimization problem is complex and require exploring a large exploration space, we dedicated a substantial budget for GA to search for optimal solutions so that they can converge. Yet, maybe increasing even more this budget may lead to better results.

5.4 External validity

Our discussion illustrates an external threat to validity. Despite our efforts, our results may not (and are not) apply to every possible CNN topology one can create. We also exhibited different behavior of NSGA-II. Other types of GA may yield different results.

6 Conclusion

Deep learning models are more and more used but they are expensive to train, especially in terms of energy consumption. While one may think that more complex models are the best as they may exhibit higher performances (*i.e.*, accuracy), this is not always the case. Depending on the context in which they will operate, seeking smaller models that use less energy (and thus are of reduced size) but perform similarly is also an interesting option. This also echoes the growing global concern of reducing carbon emissions. The results of our study highlight both the strengths and limitations of our CNN architecture search approach, particularly in balancing accuracy and energy efficiency. The optimization process using NSGA-II has demonstrated the ability to explore a diverse set of architectures, leading to promising trade-offs between performance and computational cost. However, the analysis of our predictors has revealed critical areas for improvement. One of the primary challenges observed is the inaccuracy of the predictors for both accuracy and energy consumption. Across multiple generations, discrepancies between predicted and actual values suggest that the predictors struggle to generalize beyond their training distribution. Furthermore, the presence of architectures in later generations that cannot be compiled indicates that increasing the search time (by increasing the number of generations) can be counter-productive despite reaching for new CNN architectures that seem to meet our optimisation criteria. We assume that applying multiple times the NSGA-II mutation and crossover operators may lead to inconsistencies in the description sentences of our CNN architectures since NSGA-II is not aware of constraints that are encoded into CNNGen. Thus, increasing the number of random changes on a description sentence causes more changes to lead to an architecture that cannot be compiled. Additionally, the observed performance plateau and occasional drops in accuracy suggest that the current search process may

be generating suboptimal architectures due to prediction errors. In some cases, predicted highly accurate models turned out to perform worse than expected, emphasizing the need for more robust predictive mechanisms. Similarly, energy estimations have shown inconsistencies, which could mislead the optimization process when selecting architectures based on energy efficiency. To address these limitations, several improvements are necessary. Enhancing predictors accuracy through more extensive and diverse training datasets, refined feature engineering, and advanced machine learning techniques (such as ensemble methods or uncertainty-aware models) could help mitigate out-of-distribution errors. Additionally, incorporating periodic retraining of predictors or integrating constraints on mutation and crossover could reduce the likelihood of generating architectures that are poorly estimated. Despite these challenges, the current results demonstrate the feasibility of optimizing CNN architectures with a trade-off between performance and energy efficiency. By refining our predictors and improving the search process, we can further enhance the reliability of our method, ultimately contributing to the development of more sustainable and efficient deep learning models.

7 Acknowledgement

This work was supported by Service Public de Wallonie Recherche under grant n° 2010235 – ARIAC by DIGITALWALLONIA4.AI. Gilles Perrouin is an FNRS Research Associate.

References

1. Cai, H., Chen, T., Zhang, W., Yu, Y., Wang, J.: Efficient architecture search by network transformation. In: McIlraith, S.A., Weinberger, K.Q. (eds.) Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018. pp. 2787–2794. AAAI Press (2018), <https://www.aaai.org/ocs/index.php/AAAI/AAAI18/paper/view/16755>
2. Canziani, A., Paszke, A., Culurciello, E.: An analysis of deep neural network models for practical applications. arXiv preprint arXiv:1605.07678 (2016)
3. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation* **6**(2), 182–197 (2002). <https://doi.org/10.1109/4235.996017>
4. Domhan, T., Springenberg, J.T., Hutter, F.: Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves. In: Proceedings of the 24th International Conference on Artificial Intelligence. p. 3460–3468. IJCAI’15, AAAI Press (2015)
5. Gratia, A., Liu, H., Satoh, S., Temple, P., Schobbens, P.Y., Perrouin, G.: Cnngen: A generator and a dataset for energy-aware neural architecture search. In: 32nd European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (2024)

6. Hinton, G., Deng, L., Yu, D., Dahl, G.E., Mohamed, A.r., Jaitly, N., Senior, A., Vanhoucke, V., Nguyen, P., Sainath, T.N., Kingsbury, B.: Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine* **29**(6), 82–97 (2012). <https://doi.org/10.1109/MSP.2012.2205597>
7. Jin, H., Chollet, F., Song, Q., Hu, X.: Autokeras: An automl library for deep learning. *Journal of Machine Learning Research* **24**(6), 1–6 (2023), <http://jmlr.org/papers/v24/20-1355.html>
8. Jin, H., Song, Q., Hu, X.: Auto-keras: An efficient neural architecture search system. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. pp. 1946–1956. ACM (2019)
9. Lecun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition (1998)
10. Liu, H., Simonyan, K., Yang, Y.: Darts: Differentiable architecture search (2019)
11. Miikkulainen, R., Liang, J., Meyerson, E., Rawal, A., Fink, D., Francon, O., Raju, B., Shahrzad, H., Navruzian, A., Duffy, N., et al.: Evolving deep neural networks. In: *Artificial intelligence in the age of neural networks and brain computing*, pp. 293–312. Elsevier (2019)
12. Mo, H., Iacca, G.: Accelerating evolutionary neural architecture search for remaining useful life prediction. In: *International Conference on Bioinspired Optimization Methods and Their Applications*. pp. 15–30. Springer (2022)
13. Mo, H., Iacca, G.: Evolutionary neural architecture search on transformers for rul prediction. *Materials and Manufacturing Processes* **38**(15), 1881–1898 (2023)
14. Real, E., Aggarwal, A., Huang, Y., Le, Q.V.: Regularized evolution for image classifier architecture search. In: *Proceedings of the aaai conference on artificial intelligence*. vol. 33, pp. 4780–4789 (2019)
15. Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. pp. 3645–3650. Association for Computational Linguistics, Florence, Italy (Jul 2019). <https://doi.org/10.18653/v1/P19-1355>, <https://aclanthology.org/P19-1355>
16. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G.: Llama: Open and efficient foundation language models (2023)
17. Villalobos, P., Sevilla, J., Besiroglu, T., Heim, L., Ho, A., Hobbhahn, M.: Machine learning model sizes and the parameter gap. *arXiv preprint arXiv:2207.02852* (2022)
18. Wąsowski, A., Berger, T.: Domain-Specific Languages: Effective modeling, automation, and reuse. Springer (2023)
19. Ying, C., Klein, A., Real, E., Christiansen, E., Murphy, K., Hutter, F.: Nas-bench-101: Towards reproducible neural architecture search (2019)
20. Zoph, B., Le, Q.V.: Neural architecture search with reinforcement learning. In: *International Conference on Learning Representations (ICLR)* (2017), <https://arxiv.org/abs/1611.01578>